ACTIVIDAD 8

Simulación de incidentes y ejecución de runbooks



Vértice AI Consulting · Convertimos datos en decisiones

Materia:	*Gestión de Proyectos de Inteligencia Artificial*
Actividad:	Actividad 8 · Módulo 3 / Semana 10 · Temas 19 y 20
Alumno(a):	*Eduardo Aburto Angulo*
Matrícula:	*AL07182743*
Docente:	*M. en C. Luis Ariel Vázquez Piña*
Fecha de entrega:	5 de noviembre de 2026

Declaración de Uso de Inteligencia Artificial

El alumno debe indicar de manera honesta si utilizó herramientas de Inteligencia Artificial (IA) para el desarrollo de esta actividad. Completa la siguiente tabla:

Herramienta de IA utilizada	¿La usaste?	Propósito / Descripción de uso
ChatGPT / OpenAI	Sí	ChatGPT/Codex: apoyo en programación, ejecución de pruebas, simulaciones locales, documentación y revisión visual. Las mediciones proceden de ejecuciones reales con fallas inyectadas.
Claude (Anthropic)	Antecedentes	Herramienta mencionada en actividades previas; no se atribuye ejecución nueva en Actividad 8.
Gemini (Google)	Antecedentes	Herramienta mencionada en actividades previas; no se atribuye ejecución nueva en Actividad 8.
Copilot (Microsoft)	Antecedentes	Herramienta mencionada en actividades previas; no se atribuye ejecución nueva en Actividad 8.
Otra (especifica):	Antecedentes	Herramienta mencionada en actividades previas; no se atribuye ejecución nueva en Actividad 8.

Desarrollo de Contenido

Diseño de la prueba

Preparé tres fallas reversibles dentro del servicio local, conservando intactos los pesos y el umbral del modelo. Las reglas y acciones se implementaron antes de ejecutar el laboratorio; este documento describe la ejecución y sus resultados. Utilicé un canario de 60 noticias históricas con etiquetas conocidas y solicitudes secuenciales.

Fase	Ventanas	Intervención
Normal	0 a 2	Referencia sin falla
Latencia	3 a 6	Retraso de 350 ms; corrección tras ventana 4
Calidad	7 a 10	Probabilidad 1 − p; corrección tras ventana 8
Drift	11 a 14	Texto repetido ocho veces; corrección tras ventana 12

La detección requiere dos ventanas consecutivas malas. La resolución requiere dos ventanas sanas. El hash SHA-256 del modelo se comprobó antes y después: f48c7496d5a12bd15d535721e3e83c6ef895b1ae7eaccbe7f4f4c60705bb1916. Las claves temporales se mantienen fuera de las evidencias.

Incidente de latencia

Inyecté 350 ms adicionales por predicción. El p95 máximo de esta fase fue 364.14 ms. RB01 detectó burn rate excesivo, comprobó alcance y restauró mode=normal. La detección tomó 43.457 s; el cierre tras dos ventanas sanas tomó 0.291 s desde la detección. Las 120 solicitudes lentas permanecen en el presupuesto histórico.

Acción preventiva: medir bajo concurrencia y definir ventanas temporales antes de producción. La prueba es de respuesta a retraso inyectado; no identifica un cuello de botella real de infraestructura.

Incidente de calidad

Invertí la probabilidad servida mediante 1 − p después de la inferencia. Así simulo una degradación de salida sin sustituir ni reentrenar el modelo. RB02 detectó la caída de recall o F1 con 20 etiquetas negativas por ventana, diagnosticó el modo de servicio y restauró la configuración normal.

Estado	Recall	F1
Antes	0.9500	0.9268
Durante	0.4500	0.2609
Después	0.9500	0.9268

La detección tomó 0.295 s y la recuperación verificada 0.268 s. No se atribuye la recuperación a entrenamiento ni a promoción de un modelo nuevo. Acción preventiva: añadir validación contractual de salida y mantener alerta abierta cuando no existan etiquetas suficientes.

Incidente de drift

Repetí ocho veces los textos del lote para desplazar su distribución de longitud. RB03 detectó PSI mayor a 0.20, puso en cuarentena el flujo sintético y volvió a usar la entrada de referencia. El escalamiento quedó como ticket local con reentrenamiento no ejecutado.

El PSI máximo fue 7.572; volvió a 0.000. La detección tomó 0.298 s y la recuperación verificada 0.261 s. La prueba demuestra drift de longitud; no identifica un cambio conceptual ni exige por sí sola reentrenar.

Acción preventiva: observar otras variables, comprobar origen y calidad del lote y solicitar evaluación humana antes de actualizar el modelo. Una cuarentena productiva requeriría almacenamiento separado, permisos y política de recuperación; aquí se controla el generador local.

Cronología y cierre

Caso	Inyección UTC	Detección UTC	Resolución UTC
INC-001	00:40:51.197	00:41:34.654	00:41:34.945
INC-002	00:41:34.949	00:41:35.244	00:41:35.512
INC-003	00:41:35.516	00:41:35.813	00:41:36.074

Fecha UTC de la tabla: 6 de noviembre de 2026; fecha local de entrega: 5 de noviembre. MTTD medio 14.683 s; MTTR medio 0.273 s. La corrección exacta y las seis etapas de cada runbook están en evidencias/runbook_steps.json; incidentes completos en evidencias/incidents.json.

Resultados y decisiones

Los tres incidentes finalizaron resolved y el modelo conservó su hash. La corrida demuestra el ciclo de detectar, intervenir y verificar. La disponibilidad fue 100 %, mientras el SLO agregado de latencia quedó incumplido. No oculto ese resultado: es el efecto esperado de la falla deliberada y una prueba de que el presupuesto registra el daño incluso después de la recuperación.

Mantengo como límites la escala de un host, muestra repetida y etiquetas inmediatas. En un servicio real las etiquetas tardarían en llegar y las acciones necesitarían control de concurrencia, persistencia y guardias. Mi recomendación es validar esas condiciones antes de permitir que las señales apoyen decisiones operativas de clientes.

Cómo auditar

Revisar la configuración en config/alertas.json, los procedimientos en runbooks/README.md y las mediciones originales. Ejecutar python scripts/verify_evidence.py recalcula métricas y comprueba alertas, trazas, hash y recuperación. No es necesario confiar en las cifras impresas del reporte para contrastar el resultado.